Benchmark entre orchestrateurs

meetup Python Rennes

lundi 27 avril 2026

Crédit Mutuel ARKEA

DATA office

TOSIT

The Open Source I Trust

Intervenant

DATA ENGINEER
/ DATA ARCHITECT

Jesshuan Diné

Sommaire

1 Contexte de l'étude

2 Principes généraux et comparaison

3 Benchmark de métriques *infra*

1

Contexte de l'étude

Le pourquoi du comment...

1 Un cahier des charges spécifique

Plus d'autonomie dans l'automatisation des traitements *batch*...

... à destination des utilisateurs 'métiers'

Compatibilité avec le Système d'Information d'Arkéa

Basé sur l'*open-source* et la gratuité

1 Les orchestrateurs retenus pour l'étude

Argo Workflows

Natif Kubernetes, déclarations simplifiées des workflows…

Airflow

Orchestrateur moderne de référence…

Apports de la communauté…

Windmill

UI de design, low-code, itérations dev rapides, features évoluées…

Temporal.io

Robustesse, replays, infra modulable…

À construire soi-même…

2

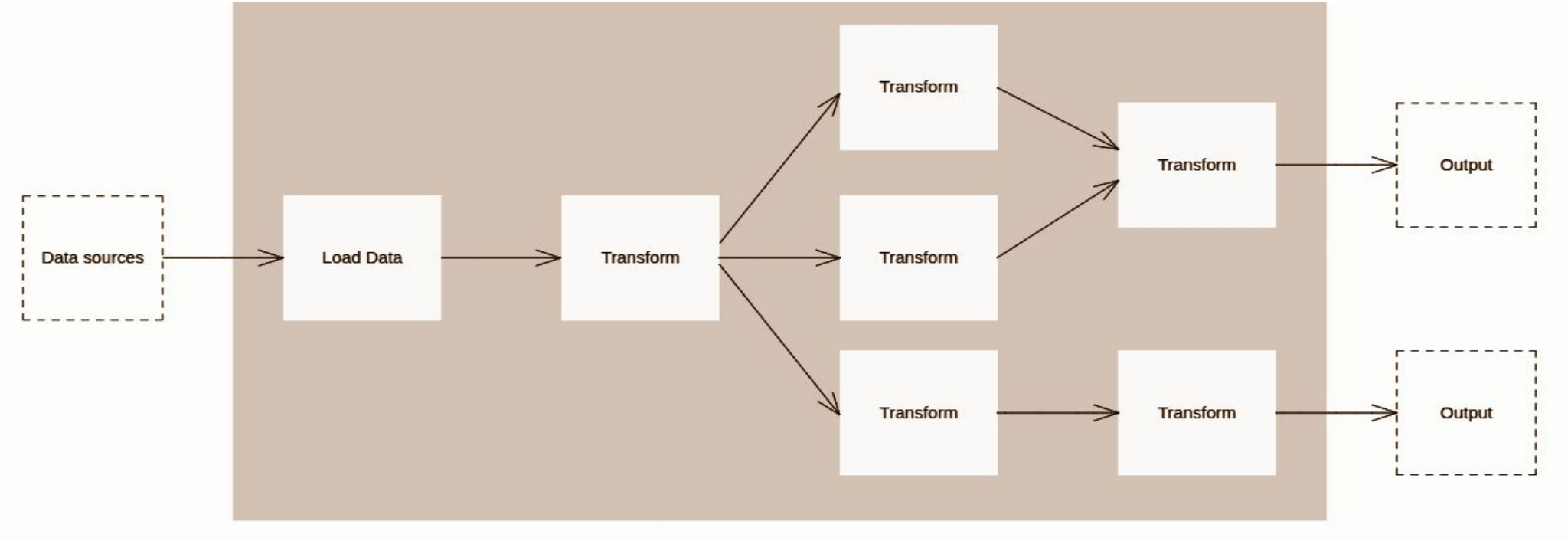
Principes généraux & comparaison

Comment comparer des chaussettes et des patates ?

4 éléments de comparaison :

- Services permanents VS workers
- Multi-tenant
- Type as-code
- UI

DAG - 'Direct Acyclic Graph'

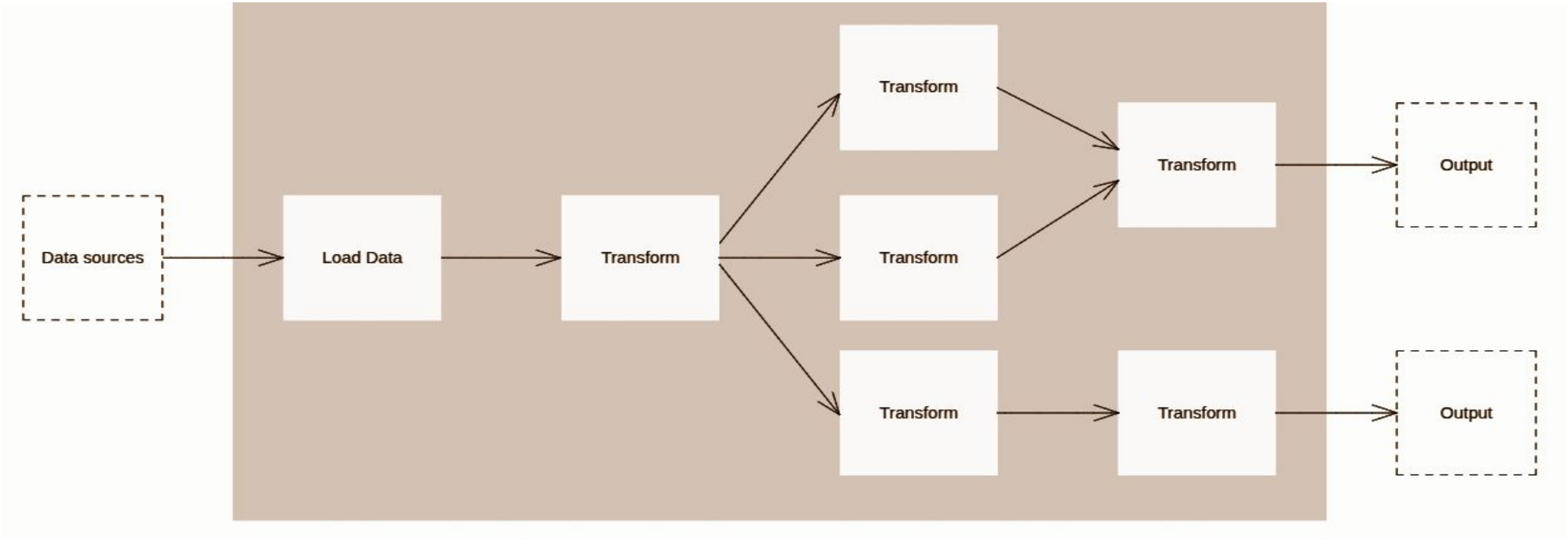


2 Principe généraux & comparaison

DAG - 'Direct Acyclic Graph'

Ordonnanceur : Quoi ? Quand ?

Orchestrateur : Quoi ? Quand ? Où ? Comment ?...

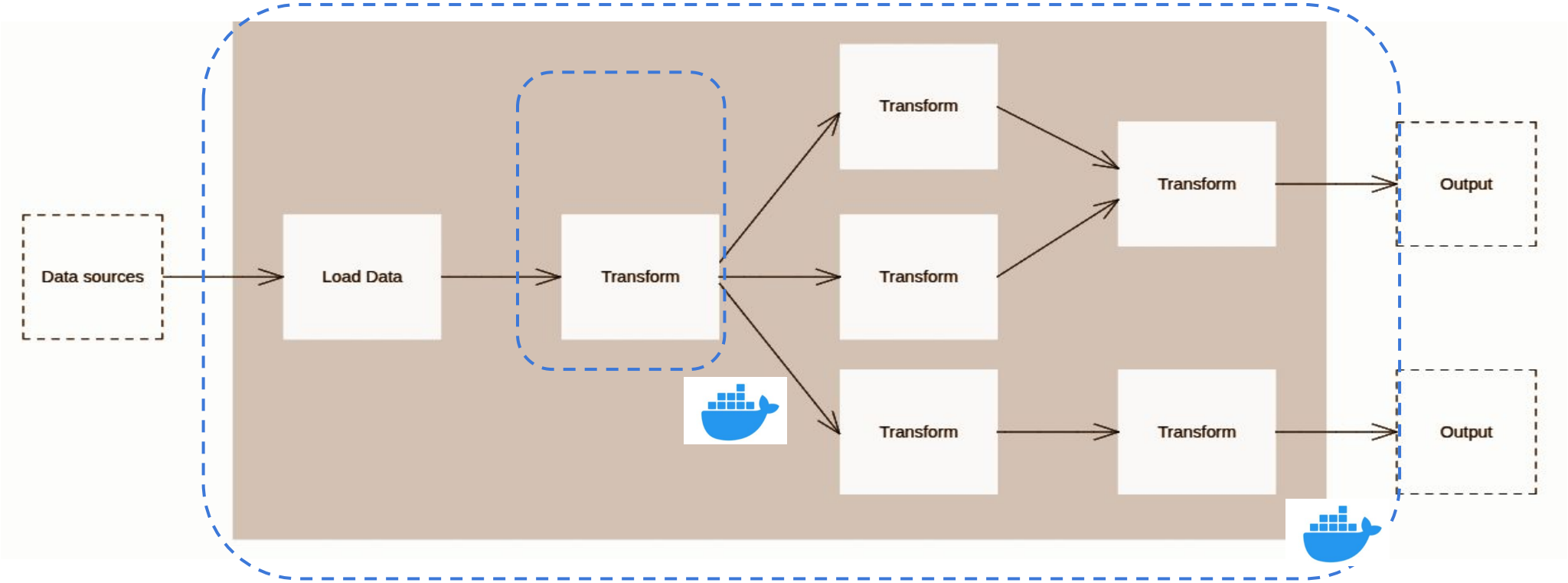


2 Principe généraux & comparaison

Ordonnanceur : Quoi ? Quand ?

Orchestrateur : Quoi ? Quand ? Où ? Comment ?...

DAG - 'Direct Acyclic Graph'



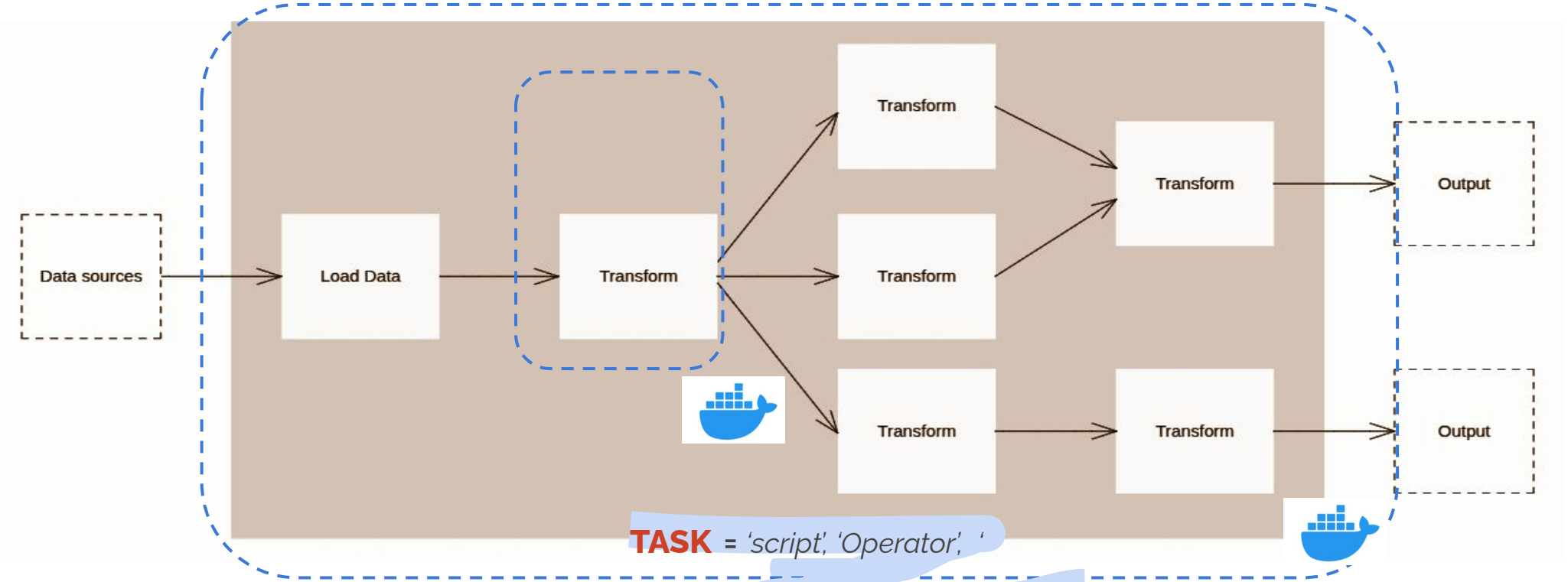
2 Principe généraux & comparaison

Ordonnanceur : Quoi ? Quand ?

Orchestrateur : Quoi ? Quand ? Où ? Comment ?...

DAG - 'Direct Acyclic Graph'

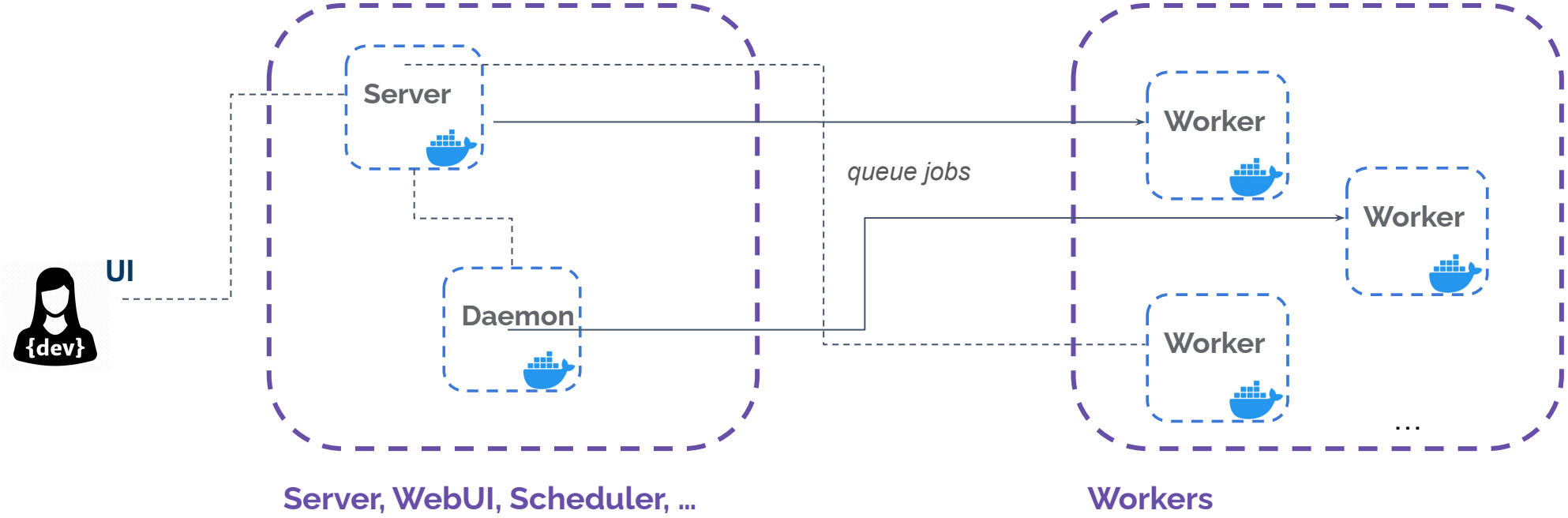
WORKFLOW = *'DAG', 'flow', etc.*



TASK = *'script', 'Operator', 'activities', 'step', 'asset', etc.*

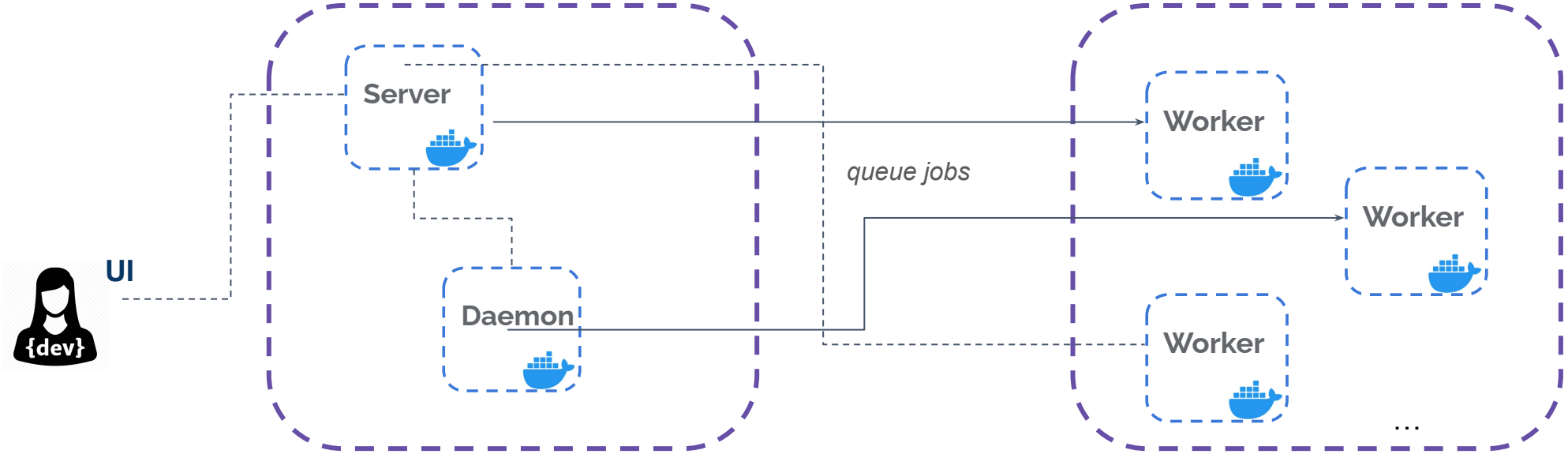
Principe généraux & comparaison

Services permanents VS workers



Principe généraux & comparaison

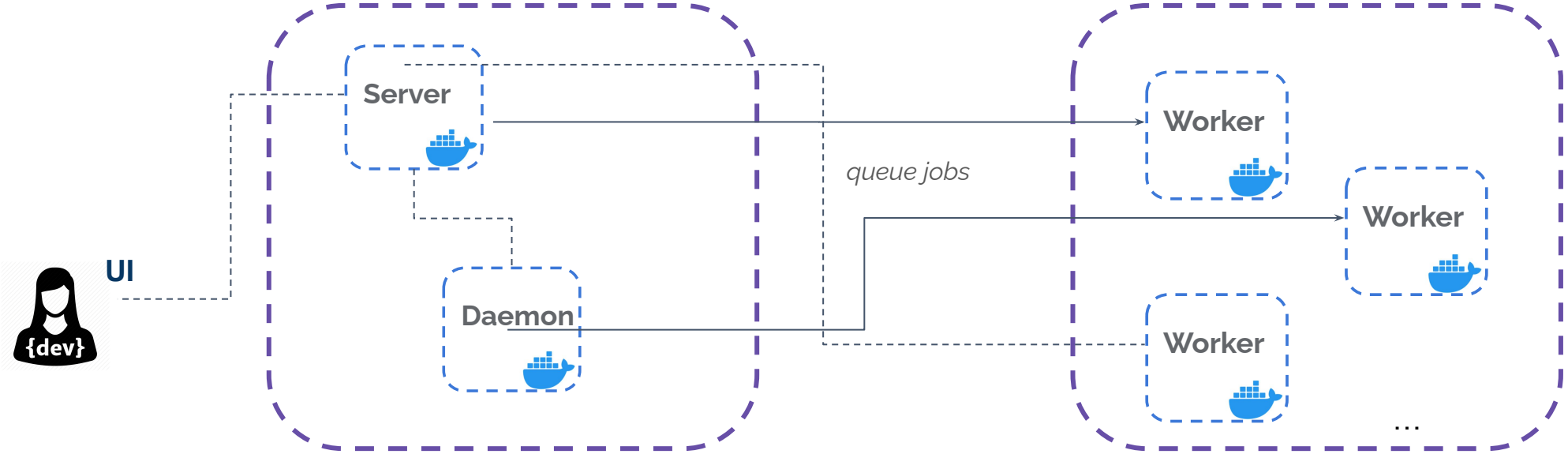
Services permanents VS workers



Server, WebUI, Scheduler, ... : permanents

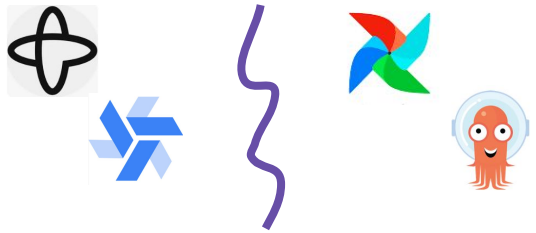
Workers : permanents OU éphémères

Services permanents VS workers



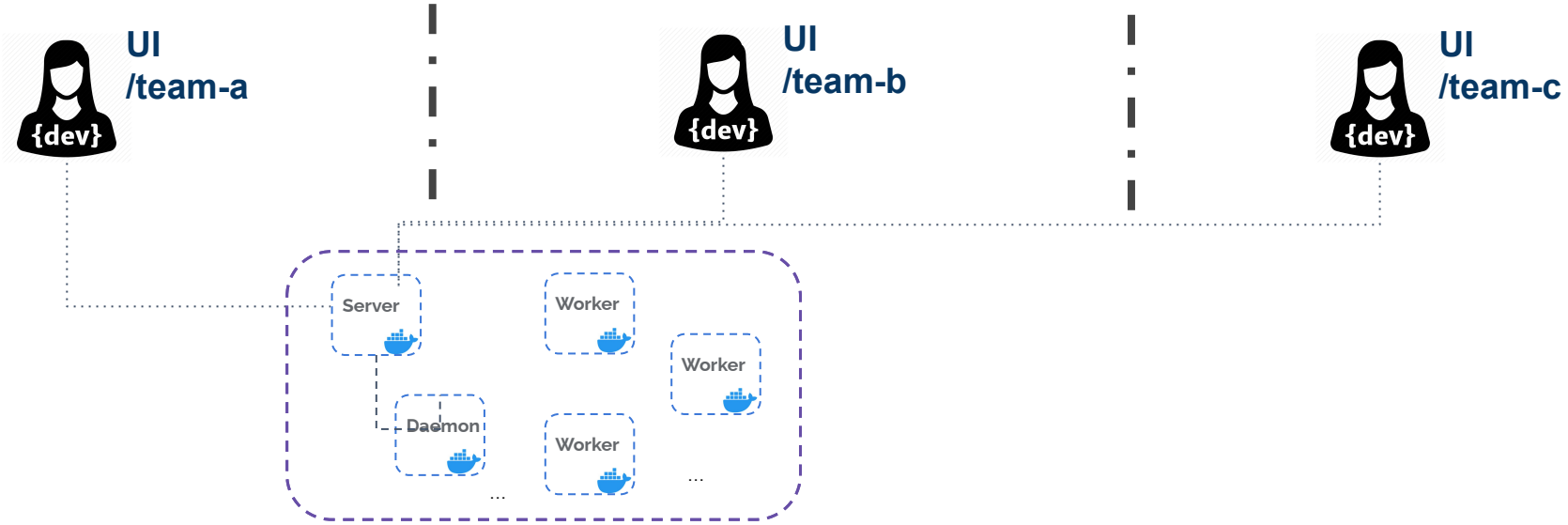
Server, WebUI, Scheduler, ... : permanents

Workers : permanents OU éphémères



Principe généraux & comparaison

Aspect *multi-tenant*

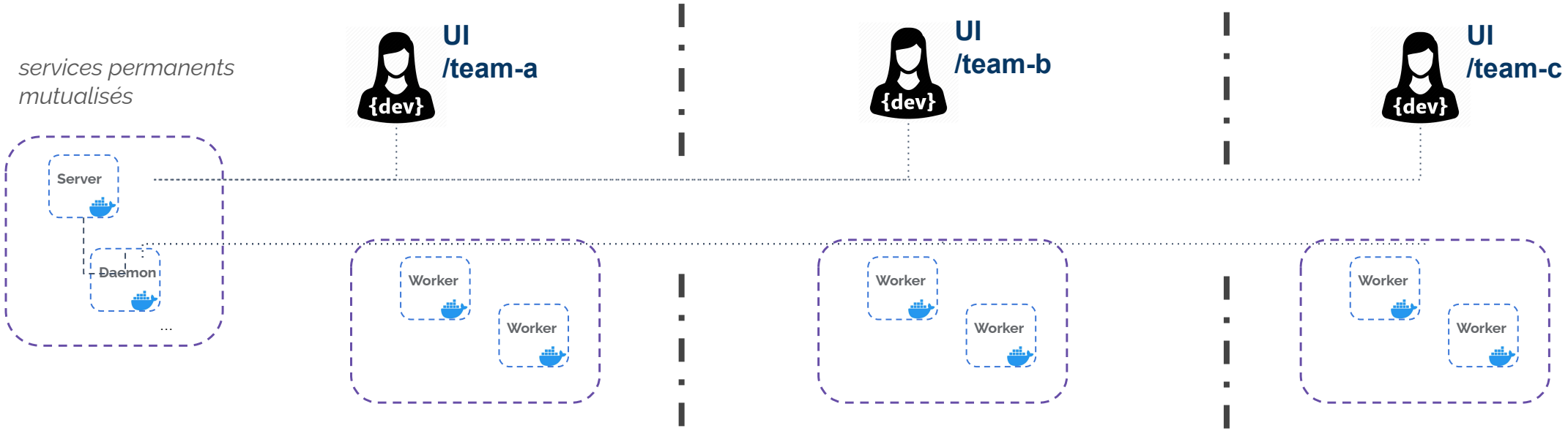


même instance ?

2 Principe généraux & comparaison

Aspect *multi-tenant*

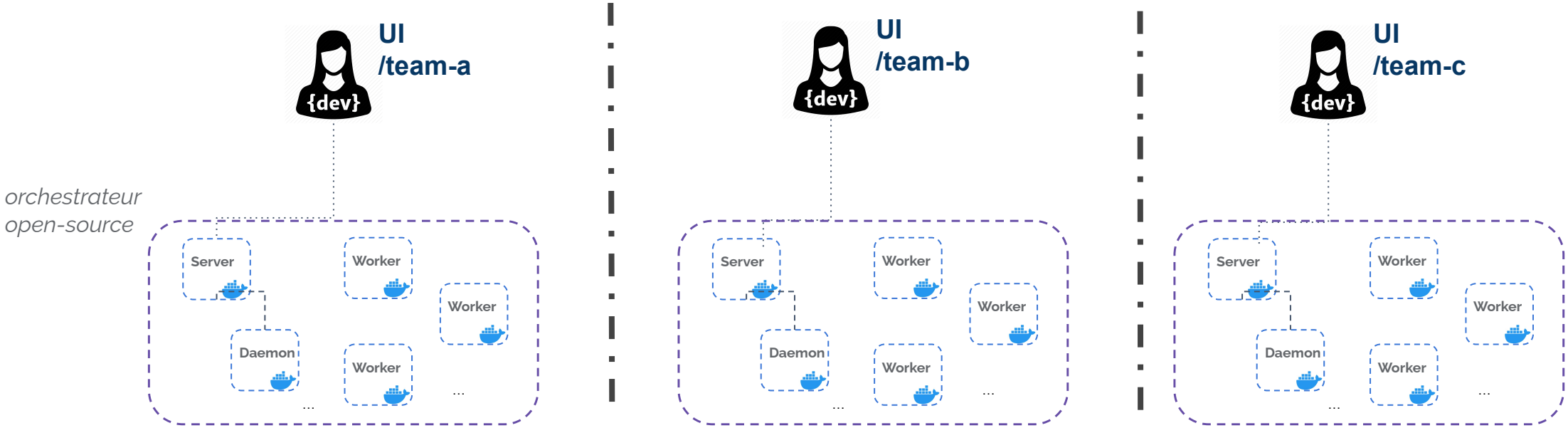
Mutualisation ?



2 Principe généraux & comparaison

Aspect *multi-tenant*

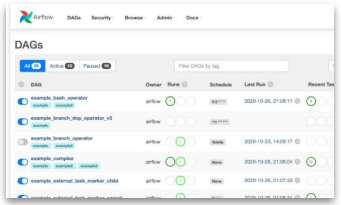
VS isolation totale ?



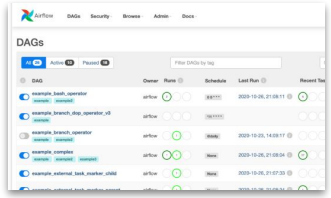
2 Exemple : Airflow

- Déploiement SI Arkéa

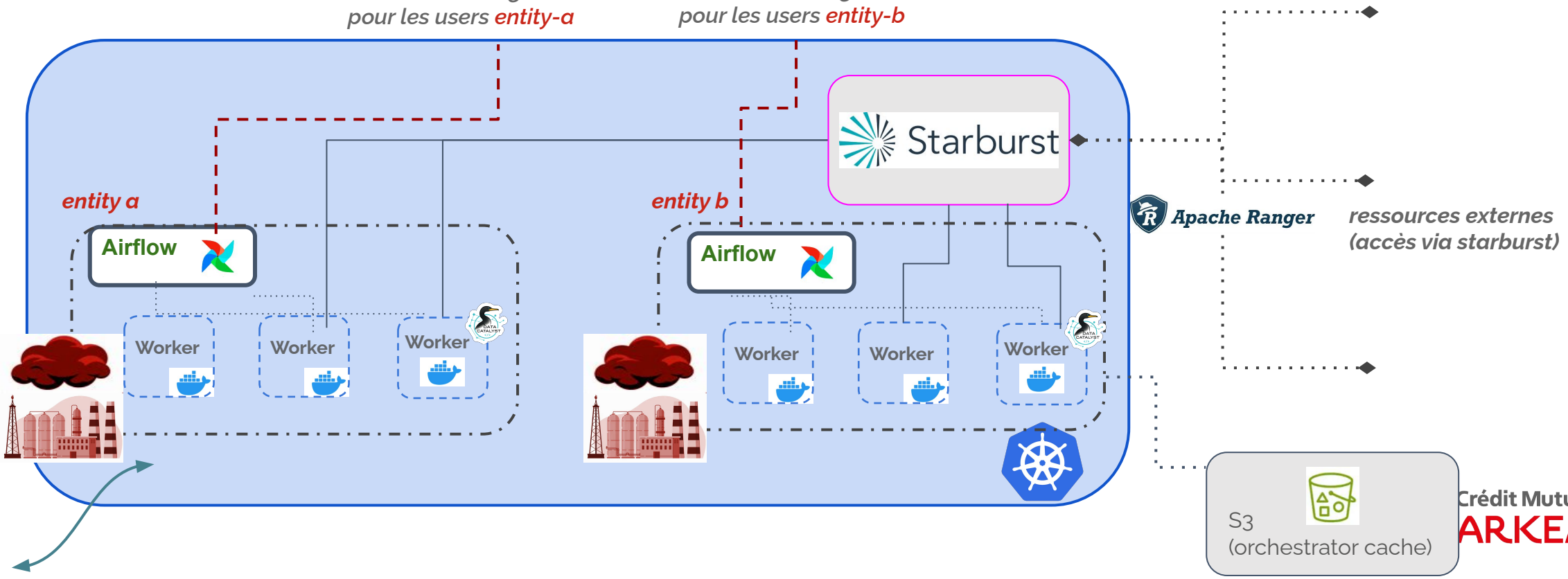
Multi-tenancy open-source



UI de monitoring pour les users entity-a



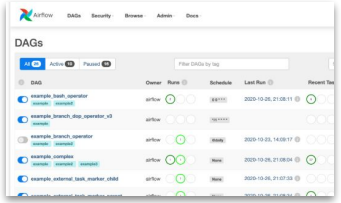
UI de monitoring pour les users entity-b



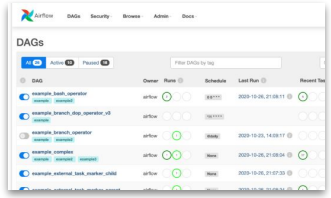
2 Exemple : Airflow

- Déploiement SI Arkéa

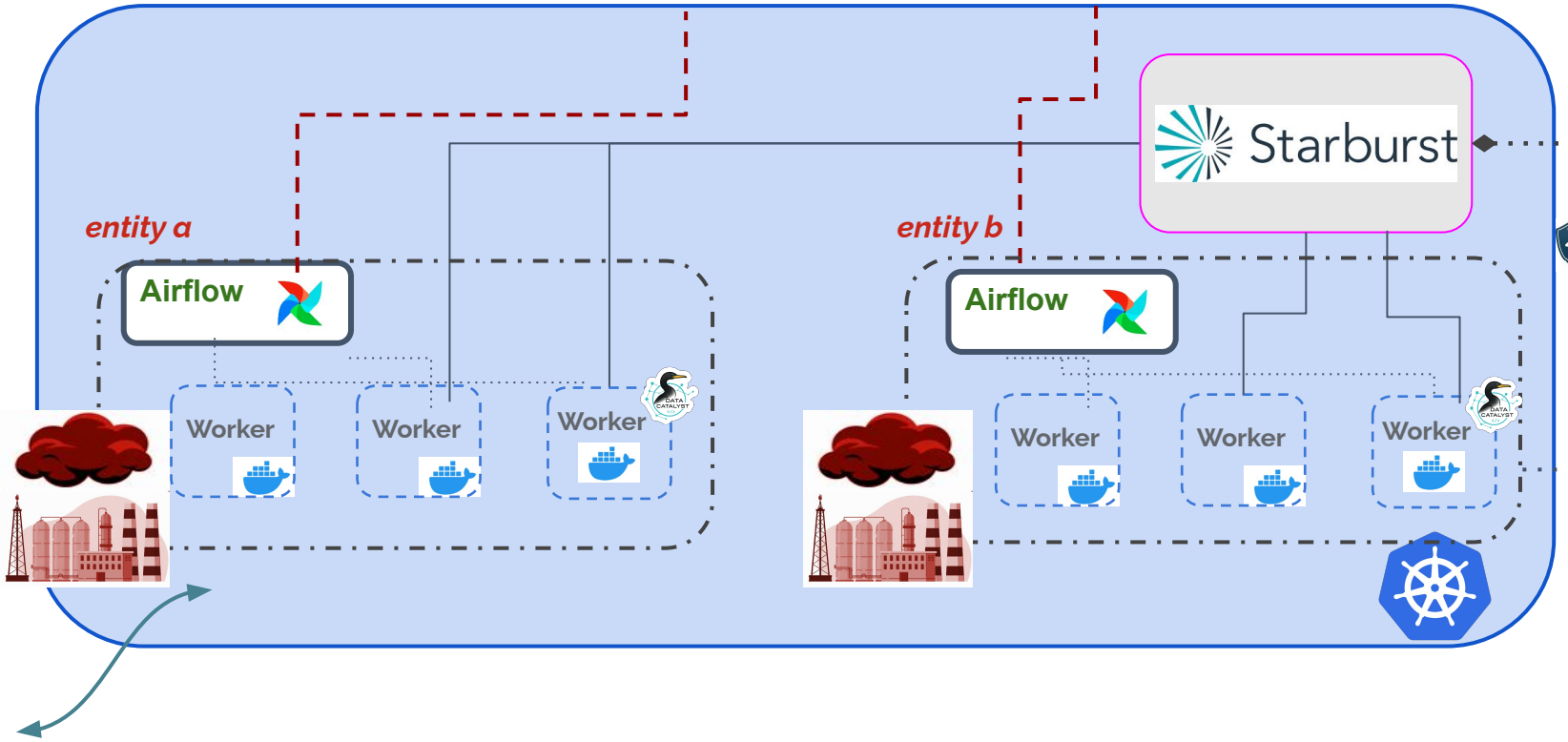
... AVEC ASTRONOMER ? AVEC GCP ?...

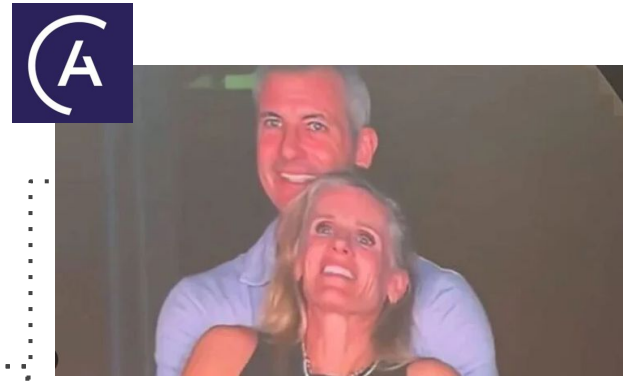


UI de monitoring pour les users entity-a



UI de monitoring pour les users entity-b





Apache Ranger

ressources externes (accès via starburst)

S3 (orchestrator cache)

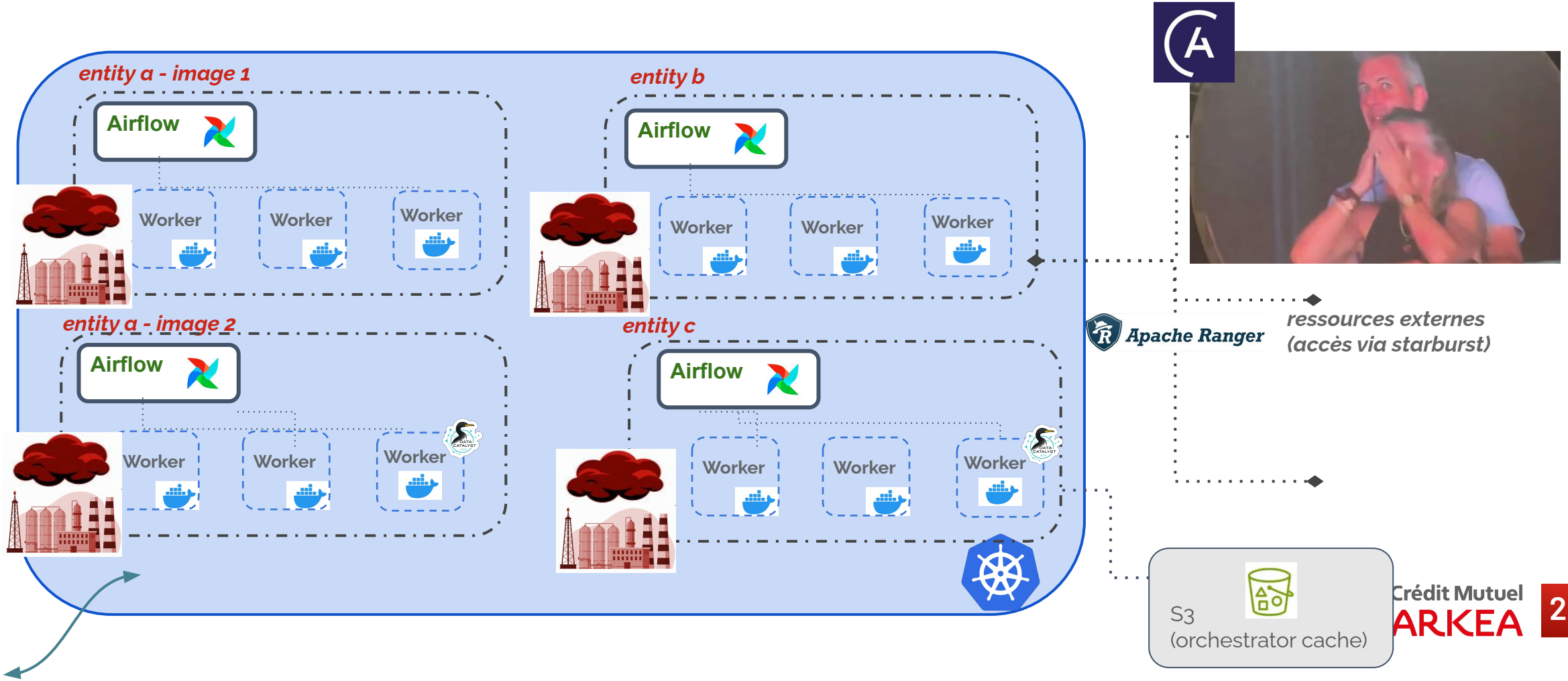
2 Exemple : Airflow

- Déploiement SI Arkéa

… c'est pire !…

car le 'multi-tenant' veut dire :

1 image de worker 'custom' différente = 1 déploiement complet !



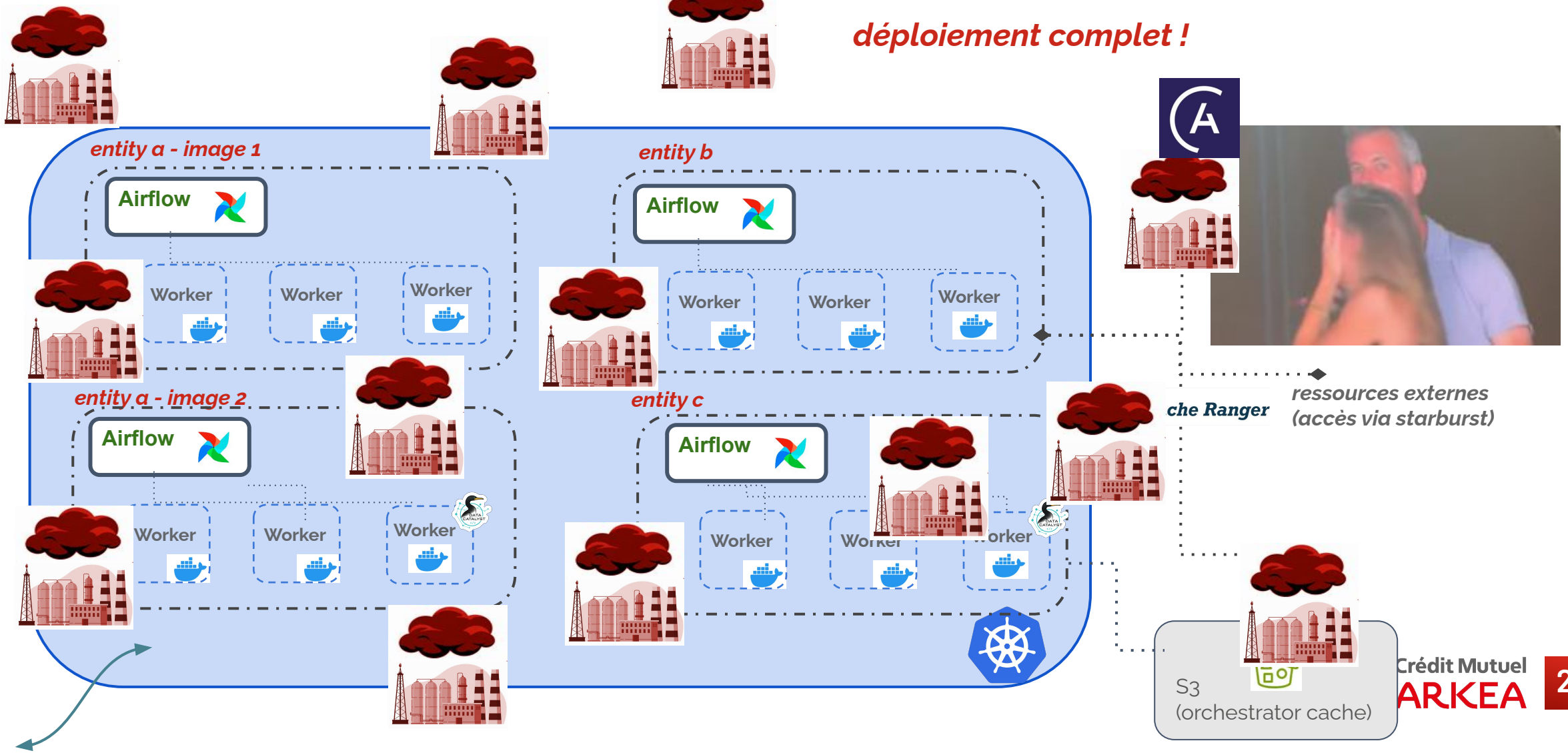
2 Exemple : Airflow

- Déploiement SI Arkéa

… c'est pire !…

car le 'multi-tenant' veut dire :

1 image de worker 'custom' différente = 1 déploiement complet !



2 Exemple : Airflow

- Déploiement SI Arkéa

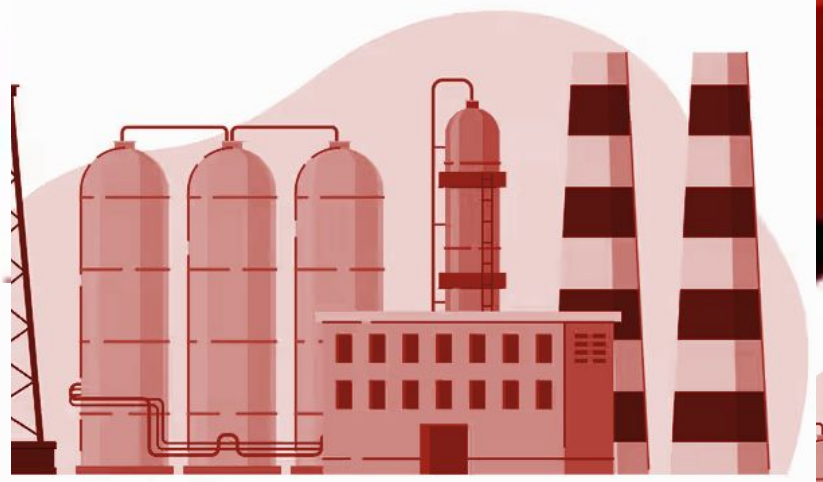
... c'est pire !...

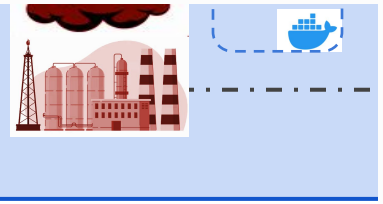
car le 'multi-tenant' veut dire :

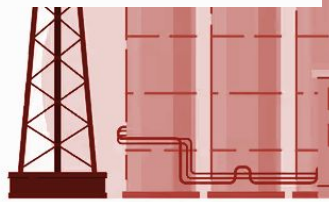
vorker 'custom' différente = 1

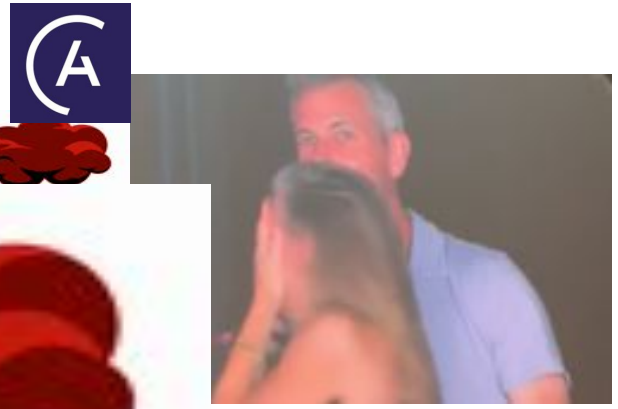
t complet !

entity a - image 1









ressources externes
(accès via starburst)

KUBA / KPOP *(Kubernetes on-prem)*

(orchestrator cache)

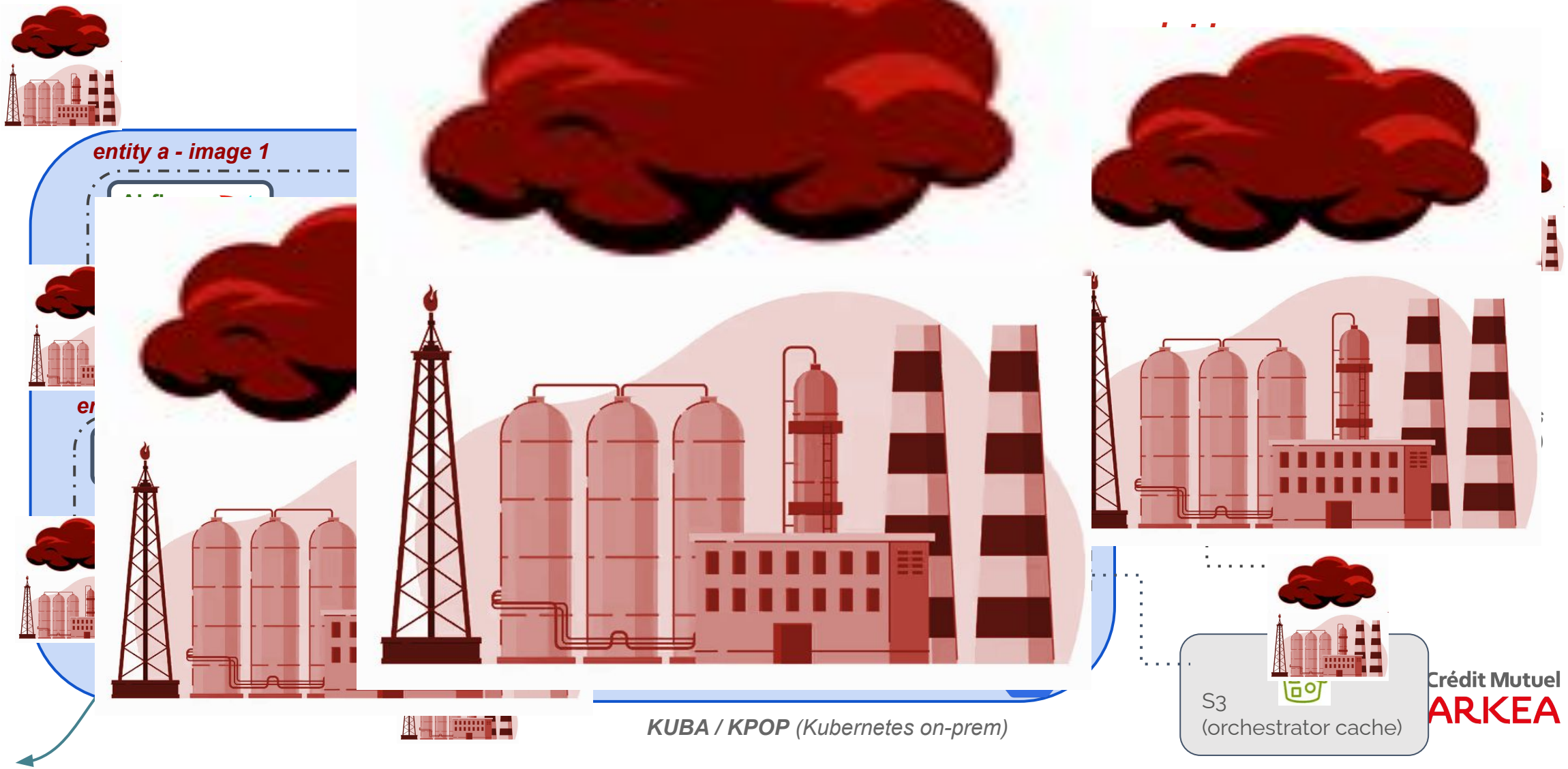
2 Exemple : Airflow

- Déploiement SI Arkéa

... c'est pire !...

car le 'multi-tenant' veut dire :

rker 'custom' différente = 1



***KUBA / KPOP** (Kubernetes on-prem)*

Crédit Mutuel ARKEA

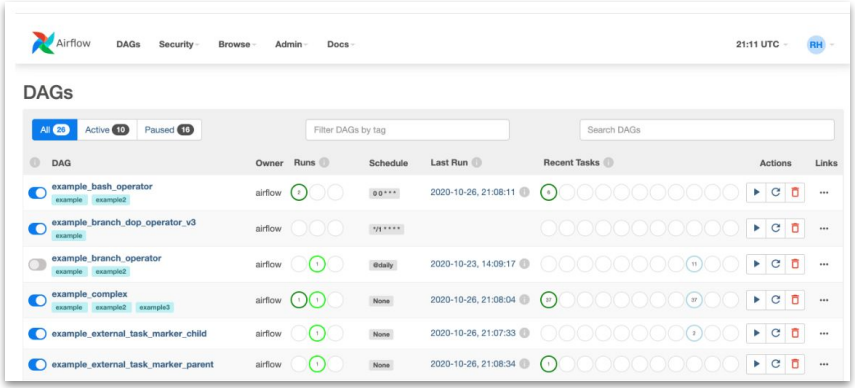
Principe généraux & comparaison

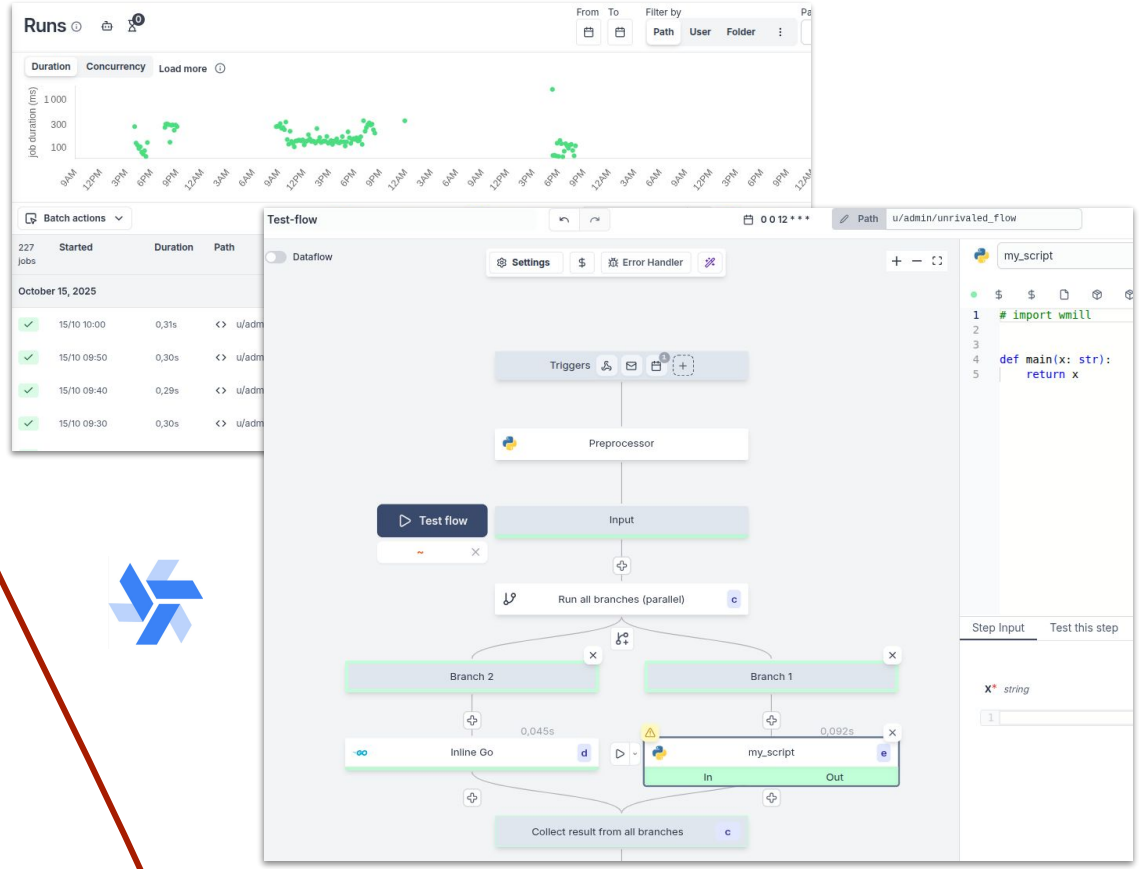
Différents types de “*as-code*”

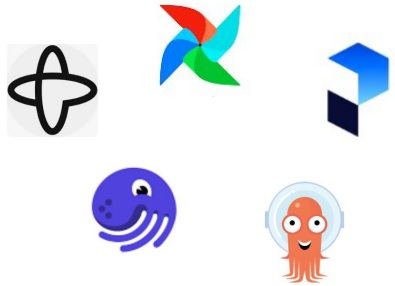
	Langage spécifique à l’orchestrateur *(orchestrator lock-in)*	*Load* dynamique (via *templates*) des définitions de scripts/flows possible …	Exemples
as-code **intégré**	🔓 🔓 🔓	✗	
full as-code	🔓 🔓 🔓	✓	
hybrid-declarative as-code	🔓	✗	

2 Principe généraux & comparaison

UI de monitoring seule VS UI de design







Principe généraux & comparaison

= à construire (dev)

= simplicité apportée par Hera

Tableau comparatif des fonctionnalités [extrait]

	Argo Workflows	Airflow	Windmill	Temporal (+ dev)
- Fonctionnalités utilisateurs 1 / 2 -				
Simplicité de déclaration des workflows	++	–	++++	++
Interface Utilisateur (UI) de *monitoring*	++	+++	+++	+++
Interface Utilisateur (UI) de *design*	✗	✗	+++	✗
Richesse et détails de visualisation des runs (tasks failed, logs, traces, etc.)	++	++	+++	+++
Catégorie *as-code*	***'declarative'***	***'as-code intégré'***	***'hybrid declarative'***	***'declarative'* (avec dev)***
Différents langages utilisateurs (dont Java)	***non as-code***	***python 100%***	✓	✓
RBAC interne à l'application/UI possible	✓ (via k8s)	✓ (via FAP)	✓	✗

Principe généraux & comparaison

= à construire (dev)

= simplicité apportée par Hera

Tableau comparatif des fonctionnalités [extrait]

	Argo Workflows	Airflow	Windmill	Temporal (+ dev)
- Fonctionnalités utilisateurs 2 / 2 -				
Compatibilité / flexibilité de mise en œuvre avec repo de code déjà existante	+	−	+	⚠ +
Compatibilité tests unitaires / POO	✗	−	+	⚠ + +
Tests simples et rapide en local sur infra simulée OU infra réelle	−	+	+ + +	⚠ +
Richesse des interface de développement	−	+	+ +	⚠ +
Simplicité des passages de variables inter-tasks & inter-workflows	−	−	+ +	⚠ + +
Accès simplifiée aux ressources (Trino, etc.)	−	+ + +	+ +	−
Simplicité de *tagging* de workers dans le code	H +	−	+ +	⚠ + +

Principe généraux & comparaison

= à construire (dev)

= simplicité apportée par Hera

$ = point supplémentaire en version payante

Tableau comparatif des fonctionnalités [extrait]

	Argo Workflows	Airflow	Windmill	Temporal (+ dev)
- Fonctionnalités utilisateurs avancées -				
Richesse / simplicité des fonctions de *loop* sur variables pour des *tasks* identiques	✚✚✚	✚✚	✚✚✚	✚✚✚
Richesse / simplicité des fonctions *human-in-the-loop, pause-suspending*...	✚	✚	✚✚✚	✚✚
Richesse / simplicité des *trigger*	✚	✚✚	✚✚ $	✚✚
Richesse / simplicité de la grammaire d'exécution (conditions logiques, etc.)	✚✚	✚✚	✚✚✚	✚✚
Richesse / simplicité des possibilités en stratégies de *retry* en cas d'erreur	✚✚	✚✚	✚✚	✚✚✚
Appels simplifiés transfert data /s3	✚✚	✚✚	✚✚ $	✚✚

2 Principe généraux & comparaison

= à construire (dev)

= niveau de consommation

$ = point supplémentaire en version payante

Tableau comparatif des fonctionnalités [extrait]

	Argo Workflows	Airflow	Windmill	Temporal (+ dev)
- Aspects d'infrastructure -				
'multi-tenant' natif	✓	✗	✓ $	✓
Scalabilité *'multi-tenant' - free open-source*	✚ ✚ ✚	✗	✚ ✚	⚠ ✚ ✚ ✚
Scalabilité *'multi-tenant'* - contrat payant $$$	*OSS uniquement*	✗	✚ ✚ ✚	⚠ ✚ ✚ ✚
Utilisation CPU/mémoire des composants permanents	🏭	🏭 🏭 🏭 🏭	🏭 🏭	🏭 🏭
Type de workers	***éphémères***	***éphémères ou persistants****	***persistants mais scalables à zéro*****	***persistants mais scalables à zéro*****
Utilisation CPU des workers		—-- *en attente*	*de benchmark* —-------	
Utilisation mémoire des workers		—-- *en attente*	*de benchmark* —-------	

** avec exécution Celery. ** avec Keda pour k8s.*

3

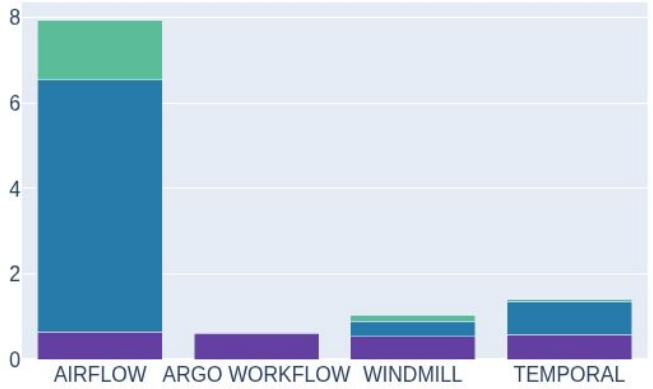
Benchmark de métriques infra

C’est là qu’on voit c’qu’ils ont dans le ventre...

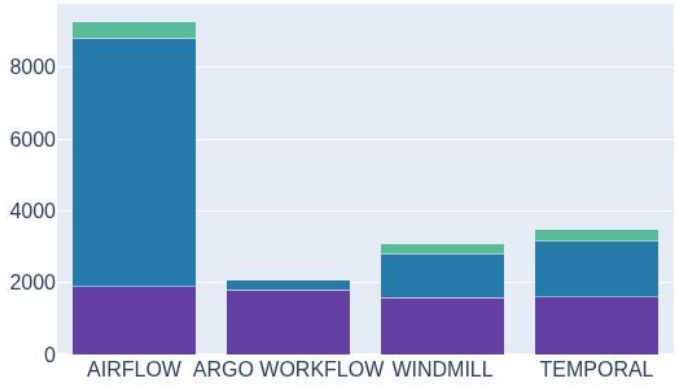
3 Benchmark & métriques

Benchmark métriques infra

Scénario ‘**NOTHING** ’ - variante “**WORKERS UP** ”



CPU *used - mean* *



Memory (Gb) - *mean*

workers

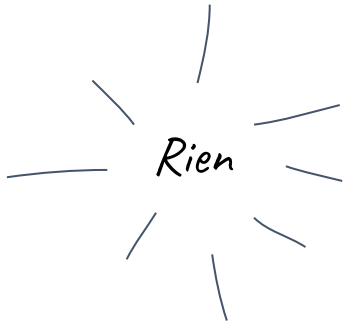
permanents

instances kube

NEW : the kubernetes part !
(offert pour le même prix !)

Bench Set-up :

- K8s kind local cluster on HP-ZBook-15-G3.
- OS: Linux - Ubuntu 24.04.
- Processor : Intel Xeon E3-1505M - 8 CPUs
- 32 Go RAM
- Workers : requests : 1 cpu / limits : 1 cpu / 6 Gb memory.



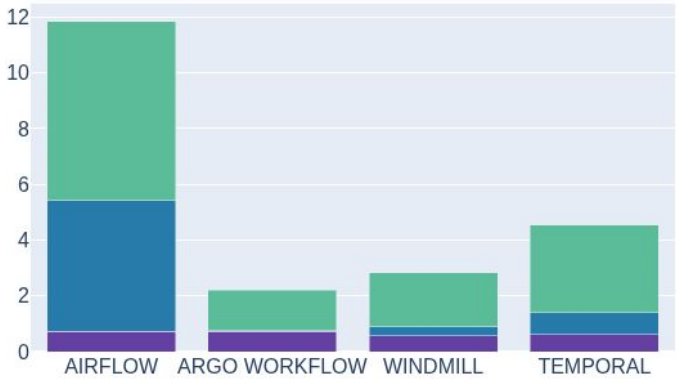
.. “ne rien faire c’est déjà faire quelque chose...”

3 Benchmark & métriques

ATTENTION : premiers résultats en cours... => à confirmer !

Benchmark métriques infra

Scénario **1 4x4** - variante "**WORKERS UP** "



CPU *used* - *mean* *

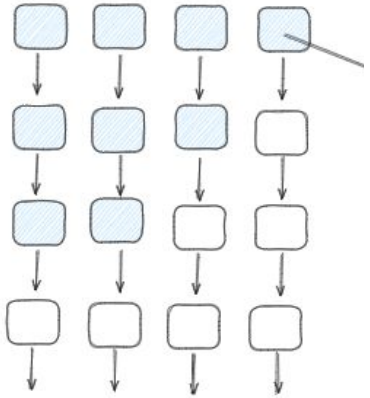
Memory (Mb) - *mean*

workers

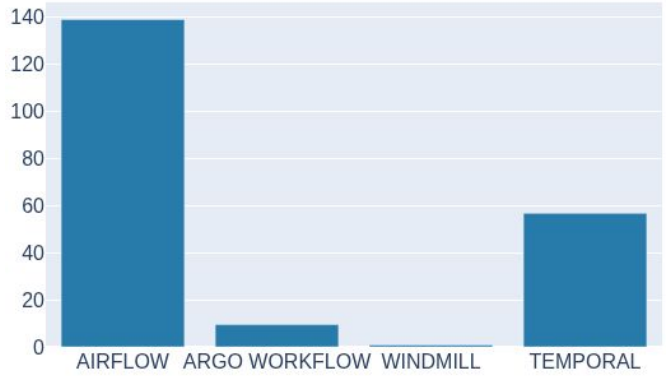
permanents

instances kube

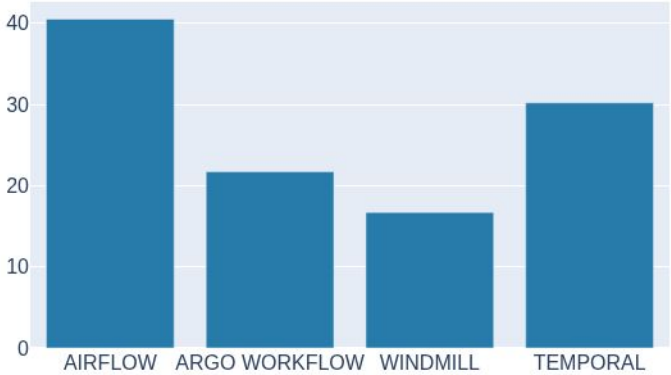
4 X 4 tasks 'cpu intensive'



approximation de Pi par méthode de monte-carlo



Workflow *starting delay* (s) - *max*



Task *execution time* (s) - *mean*

.. déclenchées toutes les 5 mins pendant 30 minutes (sur période stable)

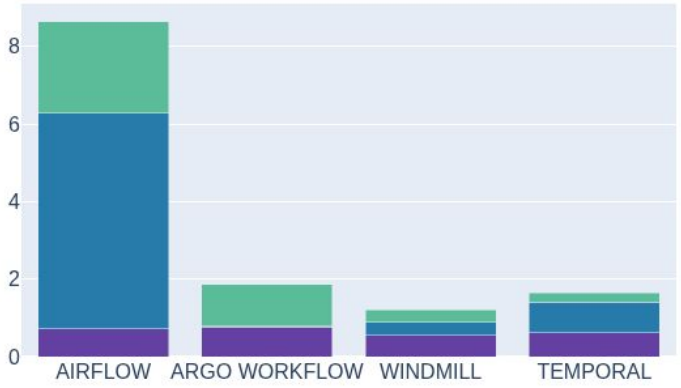
... adressés à 4 workers

3 Benchmark & métriques

ATTENTION : premiers résultats en cours... => à confirmer !

Benchmark métriques infra

Scénario 2 4x4 - variante "WORKERS UP "



CPU *used - mean* *

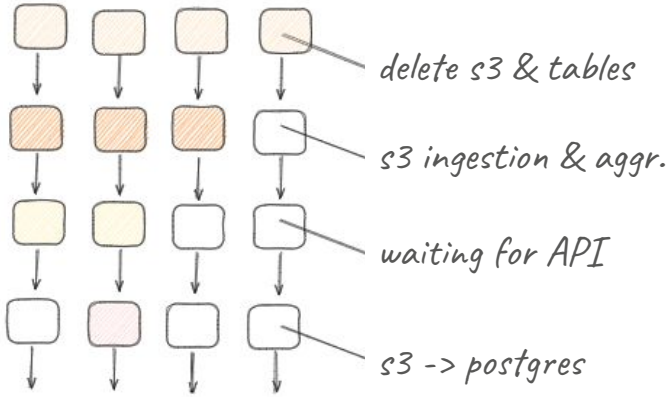
Memory (Mb) - *mean* *

workers

permanents

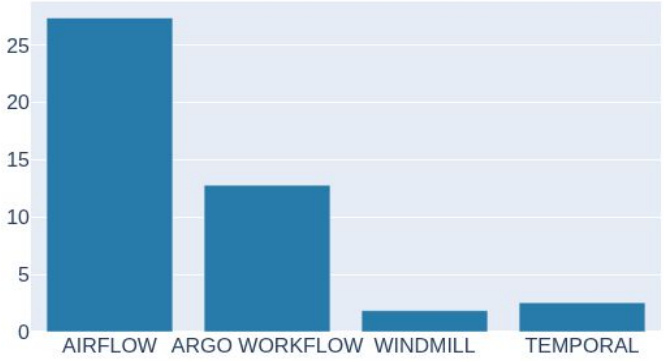
instances kube

4 X 4 tasks real world 'I/O bound'

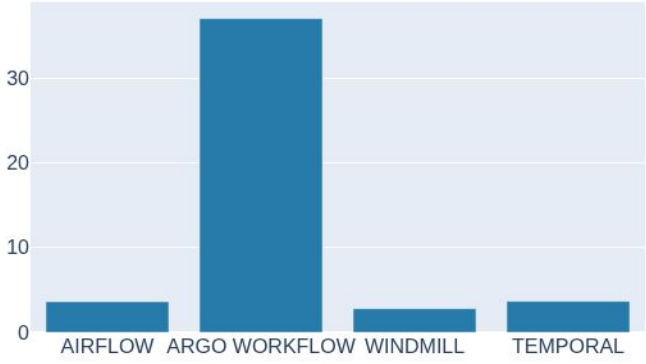


.. déclenchées toutes les 5 mins pendant 30 minutes (sur période stable)

... adressés à 4 workers



Workflow *starting delay* (s) - *max*



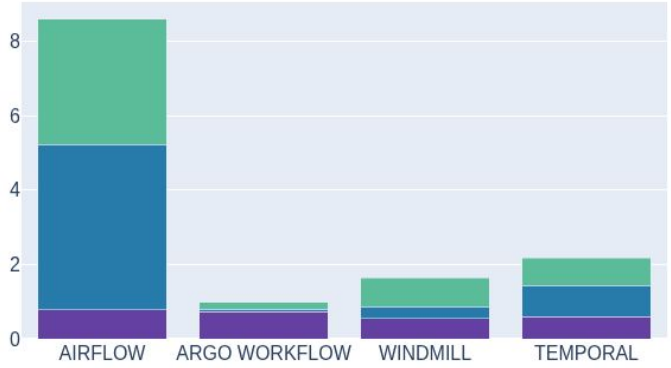
Execution time (s) - *mean - last task*

3 Benchmark & métriques

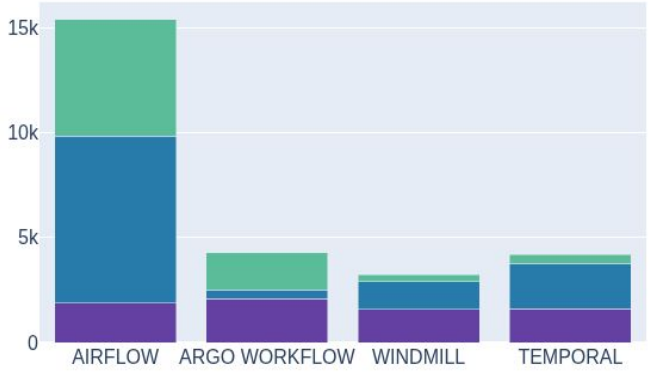
ATTENTION : premiers résultats en cours... => à confirmer !

Benchmark métriques infra

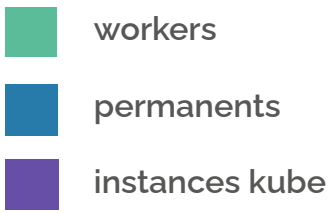
Scénario 2 12x4 - variante "WORKERS UP "



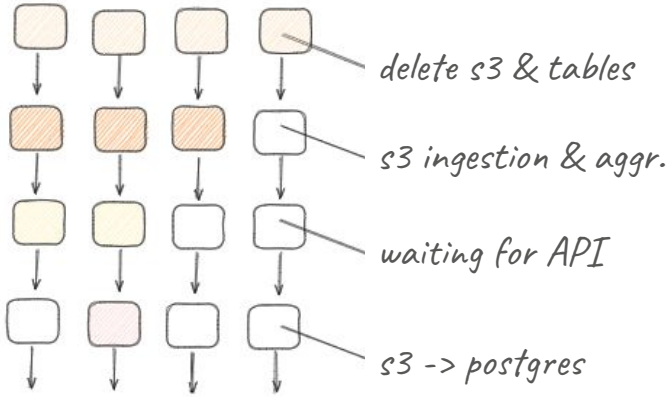
CPU used - *mean* *



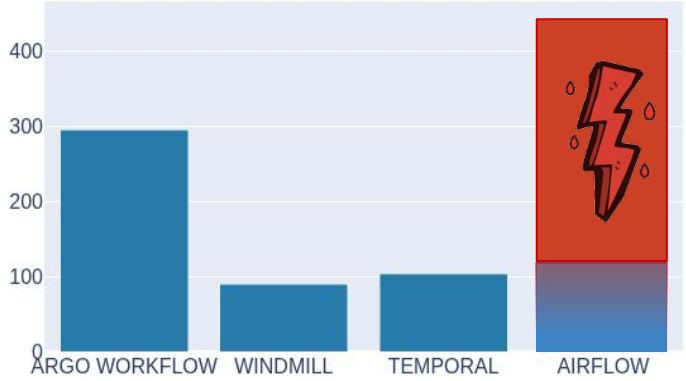
Memory (Mb) - *mean* *



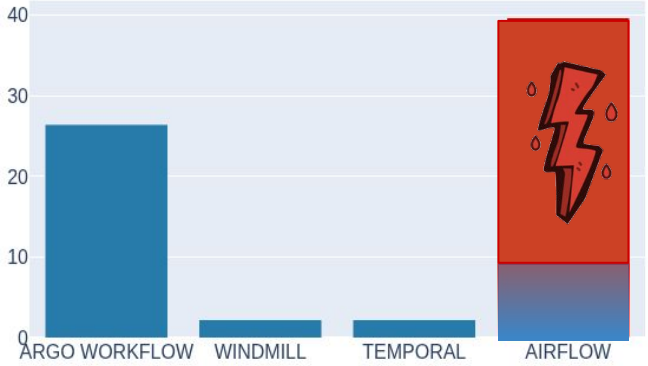
12 X 4 tasks real world 'I/O bound'



.. déclenchées toutes les 5 mins pendant 30 minutes (sur période stable)
... adressés à 4 workers



Workflow *starting delay* (s) - *max*



Execution time (s) - *mean* - *last task*

Benchmark métriques infra

Projet “Orchestrators Benchmark“

www.github.com/Jesshuan/Orchestrators-benchmark

4

Synthèse & stratégies possibles

... un orchestrateur pour les gouverner tous...

4

Synthèse & stratégies possibles

... un orchestrateur pour les gouverner tous...

Notes de synthèse & stratégies [extrait]

Nous proposons ici de retenir ***deux solutions*** :

La première plus 'court-terme ' => **Argo Workflows**

Cet orchestrateur est nativement conçu et déployé sur Kubernetes et **offre en total *open-source* toutes les fonctionnalités d'un orchestrateur digne de ce nom.**

- **Héra** **Argo Workflows en python (UI/UX)**
- **Data Catalyst.** **On capitalise sur l'existant ! (90 % des besoins)**

Synthèse & stratégies

Notes de synthèse & stratégies [extrait]

Nous proposons ici de retenir ***deux solutions*** :

La seconde plus 'moyen-terme ' => Windmill

Windmill offre une **plus-value** non négligeable

pour notre *use-case* **adressé aux *métiers* :**

faire bénéficier à l'utilisateur d'une **UI de *design* de tasks et workflows** très aboutie et intuitive, riche en fonctionnalités, pour des itérations rapides…

- robustesse INFRA ≠
- approche déclarative != *"vendor lock-in"* …
- ouverture sur l' "autom IA GEN"
- …

Synthèse & stratégies

Notes de synthèse & stratégies [extrait]

Nous pourrions alors adresser le besoin initial selon cette arborescence :

cas d'usage type
'batch analytics'
(journalier, mensuel…)

transformations 'SQL-like' ?

oui

non

DATA CATALYST + Argo Workflows

intégration fiable
et complète au sein du SI Arkéa
(data linéage, règlementation, etc…)

autre cas d'usage
(exemple : micro-batch, système d'IA, etc…)

workflow plutôt complexe ?
besoin d'itérer rapidement ?

non

oui

base de code déjà existante ?

oui

non

base de code existant + Argo Workflows

Windmill

pas de support pour les aspects réglementaires,
mise à disposition des outils seulement…

Notes de synthèse & stratégies [extrait]

Enfin, dans une perspective davantage **'long terme '**, d'autres possibilités pourraient être envisagées :

=> Développer à partir de Temporal.io un orchestrateur **'*custom-built*' (ou "fait maison")** qui permettrait aux utilisateurs de charger directement la définition de leur *workflow* sans rentrer dans la complexité de l'outil (mais travail de développement supplémentaire)

=> inclure dans cette perspective une possible interconnexion : Windmill ⇔ Temporal.io

dev itérations (UI/UX) run (exploitation)

MERCI

... des questions ?

Crédit Mutuel
ARKEA